

AI Usage Report

ResQGrid AI | IntelliCon '26 Gate 3

Author: Deshan Lakshitha

Date: October 2026

Scope: How AI tools were used to BUILD the project AND how AI operates INSIDE the product

Part A — AI Tools Used to Build the Project

A1. Antigravity IDE (Google DeepMind)

Antigravity IDE was the primary coding assistant used throughout development. It was used for:

- Generating boilerplate: FastAPI route handlers, SQLAlchemy models, Pydantic schemas, and Alembic migration files.
- Debugging: identifying root causes of CORS errors, JWT validation failures, and MapLibre worker-bundling issues.
- Architecture decisions: reviewing the AI triage ensemble design and the human-approval enforcement pattern.
- Documentation generation: producing the complete project guide (93,976 words), documentary script, pitch playbook, and these Gate 3 submission documents.
- Code review: reviewing authentication hardening, evidence-access RBAC, and dependency lock hygiene.

All AI-generated code was reviewed, tested, and modified by the developer before being committed. No AI output was merged without human verification.

A2. Alibaba Cloud Model Studio (Qwen) — build-time testing

The Qwen API was used during development to test the triage prompt pipeline with real incident narratives, calibrate the ensemble weighting between the LLM and the local lexicon engine, and validate that the Pydantic schema correctly rejected malformed AI output.

A3. GitHub Copilot (minor)

Copilot was used for inline autocomplete in the Next.js frontend — primarily for TypeScript type completions and Tailwind class suggestions. No full components were generated by Copilot without review.

Part B — How AI Operates Inside the Product

B1. AI Triage Pipeline (Core Feature)

When a dispatcher triggers triage on an incident, the following pipeline runs:

- Step 1 — Local lexicon engine: A deterministic keyword/pattern matcher classifies incident type, extracts severity signals, and estimates a confidence score. This runs offline, with no external API call.
- Step 2 — LLM triage (Qwen): If an API key is configured, the incident narrative is sent to Qwen-plus via the Alibaba Cloud Model Studio API. The prompt instructs the model to extract: `incident_type`, severity (1–5), `affected_count`, `medical_urgency` (bool), `location_description`, and `hazard_indicators`.
- Step 3 — Schema validation: The LLM response is parsed against a strict Pydantic schema. Any field that is missing, out of range, or contradictory is either rejected or set to 'unknown'. The LLM

cannot write arbitrary data to the database.

- Step 4 — Ensemble merge: The lexicon score and LLM score are weighted (configurable; default 40% lexicon / 60% LLM) and merged into a single triage result. If the LLM is unavailable, the system uses 100% lexicon output — the service does not fail.

This design ensures the AI is load-bearing (remove it and quality degrades) but not fragile (remove it and the system keeps running).

B2. Priority Scoring Engine (Deterministic, not AI)

The priority engine is entirely deterministic — no AI is involved. Six weighted factors are scored:

Factor	Weight	Source
Life risk indicator	30%	Triage output (injury/entrapment signals)
Medical urgency	25%	Triage boolean + medical_need flag
People at risk count	20%	Reported affected_count, log-scaled
Environmental hazard	10%	Hazard zone proximity + route risk score
Time sensitivity	10%	Elapsed time since report creation
Evidence confidence	5%	Number and quality of uploaded evidence items

Every priority score can be fully decomposed — the dispatcher sees exactly which factors drove the number and why. No black box.

B3. Command Assistant (LLM-powered Q&A;)

Dispatchers can ask natural-language questions (e.g., 'What are the top 3 priority incidents right now?'). The assistant queries the live database for context (incident list, resource availability, recent assignments), builds a bounded prompt, and asks Qwen to formulate a human-readable answer. If Qwen is unavailable or rate-limited, the system returns a database-formatted answer directly — it never returns an empty response or an error to the user.

B4. Image Evidence Analysis (Multimodal, Configuration-dependent)

When a multimodal model key is configured, uploaded images are sent to the LLM with a prompt asking for damage assessment signals (flood depth, structural damage, number of visible victims). The result is advisory — it is shown alongside the image but does not automatically alter the triage score. If no key is configured, the system returns a clearly labelled mock analysis.

Part C — Responsible AI Practices

- Human-in-the-loop enforced in code: No AI recommendation can dispatch a resource without explicit human approval via the API.
- Audit logging: Every AI output and every human decision is written to the audit log with a timestamp and user ID.
- Explainability: Priority scores decompose into factors. Triage results show confidence scores and source (lexicon vs LLM).
- Graceful degradation: The system is designed to work without any AI key — quality is lower but the workflow is intact.
- No PII in prompts: Prompts are constructed from incident metadata, not from user personal data.
- Schema validation as a safety boundary: The LLM cannot produce output that bypasses schema constraints.